

Database Design & User Flows

Version 1.0.0

March 24, 2026

Architecture Reference Guide

CONFIDENTIAL -- Internal Use Only

ChoreRobotics, Inc. -- All Rights Reserved

Table of Contents

Part 1 User Journey Map

Part 2 Screen-by-Data Matrix

Part 3 Database Design -- What Needs Firebase

Part 4 What Does NOT Need a Database

Part 5 Data Lifecycle

Part 6 Connection Modes & Data Routing

Part 7 Security & Permissions

Part 8 Attachment System

Part 9 Offline Strategy Decision Tree

Appendix A Complete File-to-Screen Mapping

Appendix B Firestore Index Requirements

Appendix C Data Retention Policies

Appendix D Cost Summary (Firebase Free Tier)

Confidential

OVERVIEW

System Overview for New Developers

ChoreRobotics builds an autonomous outdoor robot that mows lawns, plows snow, and collects leaves. The Flutter mobile app controls the robot through three communication paths:



Key architectural principle: The robot works without the internet. The app works without the internet. Firebase adds persistence, remote access, and push notifications -- but is never a hard dependency for operating the robot.

The three entities:

- **Mobile App** -- Flutter, runs on user's phone. Controls robot, displays data, edits zones/schedules.
- **Robot** -- Jetson Nano running ROS2/Python. Has BLE GATT server, WiFi WebSocket server, and (when internet is available) Firebase Admin SDK.
- **Firebase** -- Cloud persistence layer. Firestore for structured data, Realtime Database for live telemetry, Storage for firmware binaries and coverage maps.

PART 1

USER JOURNEY MAP

Journey 1: New User -- First-Time Setup

```
INSTALL APP
    |
    v
/auth (AuthScreen)
    |  User sees: Logo, email/password fields, Google/Apple sign-in,
    |             "Create Account" button, "Continue as Guest" link
    |
    |  DATA WRITTEN: None yet
    |  DATA SOURCE: None
    |  OFFLINE: Screen loads fine, but sign-in requires internet
    |           (except "Continue as Guest")
    |
    +--> [Create Account] or [Sign In] or [Google/Apple]
    |
    |  DATA WRITTEN:
    |    - Firebase Auth creates user account (cloud)
    |    - Firestore: users/{userId} document created with:
    |        email, displayName, settings (defaults), notifications (defaults),
    |        robotIds: [], accountStatus: "active", createdAt, updatedAt
    |  DATA SOURCE: Firebase Auth + Firestore
    |
    v
/onboarding/scan (BleScanScreen)
    |  User sees: "Find Your Robot" title, BLE scanning animation (pulse rings),
    |             info card explaining pairing, "Can't find your robot?" link
    |
    |  DATA READ: BLE scan results (device names, signal strength)
    |  DATA SOURCE: Phone Bluetooth hardware -- no database involved
    |  OFFLINE: Works perfectly -- BLE scanning is local
    |
    |  After ~2.5 seconds, robot appears:
    |    "ChoreBot Alpha" with signal strength bars
    |
    +--> [Pair This Robot]
    |
    v
/onboarding/pair (PairingScreen)
    |  User sees: Robot identity card (name, UUID: CHORE-A1B2-C3D4, BLE signal),
    |             UUID verification prompt ("check the label on your robot"),
    |             Attachment detection card (e.g., "Mower Blade Assembly"),
    |             6-digit pairing code entry field
    |
    |  DATA READ:
    |    - Robot UUID: from BLE advertisement data (ROBOT-ONLY)
    |    - Attachment type: from BLE GATT characteristic (ROBOT-ONLY)
    |    - BLE signal strength: from phone hardware (LOCAL-ONLY)
    |  DATA WRITTEN:
    |    - Pairing code sent to robot via BLE write characteristic (ROBOT-ONLY)
    |    - Robot responds with confirmation via BLE notify (ROBOT-ONLY)
    |  DATA SOURCE: All data from BLE -- zero Firebase involvement
    |  OFFLINE: Works perfectly -- pairing is entirely local
    |
```

```
    +--> [Pair This Robot] (after entering 6-digit code)
    |
    v
/onboarding/wifi (WifiSetupScreen)
    |  User sees: Success banner ("Paired securely with ChoreBot Alpha"),
    |             explanation of why WiFi helps, list of available WiFi networks
    |             (scanned by the robot and sent to app via BLE), password field,
    |             "Skip -- use direct connection only" link
    |
    |  DATA READ:
    |    - WiFi network list: robot scans and sends via BLE (ROBOT-ONLY)
    |  DATA WRITTEN:
    |    - WiFi SSID + password: sent to robot via BLE write (ROBOT-ONLY)
    |    - Robot connects to WiFi and confirms success via BLE notify
    |  DATA SOURCE: All from BLE to robot
    |  OFFLINE: Works perfectly -- WiFi setup goes through BLE
    |
    +--> [Connect Robot to Wi-Fi] or [Skip]
    |
    v
/onboarding/complete (SetupCompleteScreen)
    |  User sees: Green checkmark animation, robot summary card
    |             (name, connection mode, WiFi network, battery 100%, attachment),
    |             "What's Next" checklist (set up yard, choose pattern, start job)
    |
    |  DATA WRITTEN (this is the key moment for Firebase):
    |    - Firestore: robots/{serialNumber} document created:
    |        serialNumber: "CR-MK1-00001"
    |        model: "CHOREROBOT-MK1"
    |        ownerId: <user's Firebase Auth UID>
    |        name: "ChoreBot Alpha"
    |        firmwareVersion: "1.0.0"
    |        status: "online"
    |        attachment: { type: "mower_blade", status: "connected" }
    |        wifiConfig: { ssid: "HomeWiFi-5G", configuredAt: <now> }
    |        pairedAt: <now>
    |        createdAt: <now>
    |    - Firestore: users/{userId}.robotIds updated to include new robot ID
    |    - Realtime DB: _robot_owners/{robotId} set to userId (via Cloud Function)
    |  DATA SOURCE: Robot info from BLE, written to Firebase
    |  OFFLINE: If no internet, Firestore SDK queues the writes.
    |           The robot still works locally. Writes sync when internet returns.
    |
    +--> [Go to Dashboard]
    |
    v
/dashboard (DashboardShell -> DashboardScreen)
    |  User is now in the main app. See Journey 2.
```

Journey 2: Returning User -- Daily Operation

```
OPEN APP
    |
    v
/splash (SplashScreen) [after Firebase Auth integration]
    |  Checks FirebaseAuth.authStateChanges()
    |
    |  DATA READ: Firebase Auth cached credentials (LOCAL)
    |  OFFLINE: Auth token cached locally -- app opens without internet
    |
    +--> [Has token + has robots] --> /dashboard
    +--> [Has token + no robots] --> /onboarding/scan
    +--> [No token] --> /auth
    |
    v
/dashboard (DashboardScreen)
    |  User sees:
```

```
    |    - Connection bar (Local/Remote/Offline + battery %)
    |    - Robot status card (state: Idle/Mowing/Charging/Error, attachment, battery)
    |    - Quick actions: [Mow] [Plow] [Collect]
    |    - Live metrics (battery cycles, signal, blade, speed, temp)
    |    - Recent activity log (last 3-5 events)
    |
    |  DATA READ:
    |    Connection bar    --> connectionProvider (local BLE/WiFi state)
    |    Robot status      --> Firestore robots/{id} (cached) + RTDB telemetry
    |    Battery %         --> RTDB robots/{id}/telemetry/battery_pct
    |    Live metrics      --> RTDB telemetry (all fields)
    |    Recent activity   --> Firestore robots/{id}/task_history (last 5, cached)
    |    Attachment info   --> Firestore robots/{id}.attachment (cached)
    |
    |  OFFLINE BEHAVIOR:
    |    - Has BLE/WiFi: Full live data from robot
    |    - Has internet only: Dashboard via Firestore cache + RTDB telemetry (5s delay)
    |    - No connection: Shows last cached state. "Offline" badge. Retry every 5s.
    |
    +--> [Tap "Mow"]
    |
    v
/mowing (MowingScreen)
    |  User sees:
    |    - Zone summary (name, area m2, no-go count, "Ready" badge)
    |    - Pattern selection grid (Straight, Diagonal, Checker, Spiral)
    |    - Stripe angle slider (0-180 degrees)
    |    - Pass overlap slider (0-30%)
    |    - Estimated time + battery needed
    |    - [Start Mowing] button
    |
    |  DATA READ:
    |    Zone info         --> Firestore robots/{id}/zones (cached in Hive)
    |    Attachment specs   --> Firestore robots/{id}/attachments (blade height range)
    |    Battery current    --> RTDB telemetry/battery_pct (can we complete this job?)
    |
    |  DATA WRITTEN ON START:
    |    If BLE/WiFi available (preferred):
    |      - Send start_task command directly to robot via BLE/WiFi
    |      - Command includes: taskType, zoneId, pattern, angle, overlap, bladeHeight
    |    If cloud-only (no local connection):
    |      - Create Firestore robots/{id}/commands/{auto-id}:
    |          type: "start_task"
    |          payload: { taskType: "mow", zoneId: "zone1", parameters: { ... } }
    |          status: "pending"
    |          createdAt, expiresAt, createdBy
    |
    |  OFFLINE BEHAVIOR:
    |    - Has BLE/WiFi: Works perfectly. Command goes direct to robot.
    |    - Has internet only: Cloud command sent. Robot picks it up in 1-5 seconds.
    |    - No connection: [Start Mowing] button disabled. Shows "Connect to robot first."
    |
    +--> Task starts, user returns to dashboard
    |
    v
/dashboard (during active task)
    |  Dashboard now shows:
    |    - Status card: "Mowing" state, progress %, ETA countdown
    |    - Live metrics update in real-time (speed, heading, battery draining)
    |    - [Pause] and [Return to Dock] buttons appear
    |
    |  DATA READ (continuous):
    |    Robot state       --> RTDB telemetry/state == "working"
    |    Progress          --> RTDB telemetry/active_task_id (task running)
    |    Speed, heading    --> RTDB telemetry (0.2Hz via cloud, 10Hz via BLE)
    |    Battery drain     --> RTDB telemetry/battery_pct
    |
    +--> Task completes (robot reports via BLE or Firestore)
    |
    |  DATA WRITTEN BY ROBOT:
    |    - Firestore: robots/{id}/task_history/{auto-id}:
    |        taskType: "mow", zoneId, zoneName, status: "completed"
    |        startedAt, completedAt, durationMinutes, areaCoveredSqM
    |        batteryStartPct, batteryEndPct, batteryUsedPct
    |        taskResults: { stripesCompleted, cuttingHeightMm, pattern }
    |    - Storage: robots/{id}/coverage/{taskId}.json (GPS path for coverage map)
    |    - Firestore: robots/{id} status updated to "docked" or "idle"
    |    - Cloud Function: sends push notification "Front Yard mowing complete"
    |
    v
USER DONE -- closes app or checks history
```

Journey 3: Multi-Robot User -- Fleet Management

```
/dashboard
    |  App bar shows current robot name: "ChoreBot Alpha"
    |
    +--> [Tap robot name in app bar] OR [Go to Profile -> Manage Fleet]
    |
    v
Bottom sheet (or /fleet screen)
    |  User sees list of all robots from Firestore:
    |    - ChoreBot Alpha (active, online, 78%, Mower Blade)
    |    - ChoreBot Beta (offline, 42%, Snow Plow)
    |    - ChoreBot Gamma (charging, 95%, Leaf Collector)
    |    - [+ Add New Robot] button
    |
    |  DATA READ:
    |    Robot list        --> Firestore users/{uid}.robotIds array
    |    Robot details     --> Firestore robots/{id} for each robot in array
    |    Each robot status --> Firestore robots/{id}.status + lastSeenAt
    |  DATA SOURCE: Firestore (cached after first read)
    |  OFFLINE: Shows last cached fleet state. "Offline" labels may be stale.
    |
    +--> [Tap different robot]
    |
    |  LOCAL STATE CHANGE:
    |    - Update active robotId in local state (Riverpod/BLoC)
    |    - All screens reload data for the new robot
    |    - Telemetry stream switches to new robot's RTDB path
    |    - BLE connection: attempt to discover and connect to new robot
    |
    +--> [+ Add New Robot]
    |
    v
/onboarding/scan (same as first-time setup, Journey 1 step 2)
```

Journey 4: Attachment User -- Browse, Purchase, Register

```
/dashboard
    |  Dashboard shows current attachment: "Mower Blade attached"
    |
    +--> [Tap attachment info] or [Navigate to attachment store]
    |
    v
/attachments/store (AttachmentStoreScreen) [Phase F4]
    |  User sees:
    |    - YOUR ATTACHMENTS: registered attachments for current robot
    |        Mower Blade (Connected), Snow Plow (Stored)
    |    - AVAILABLE TO PURCHASE: catalog items
    |        Edge Trimmer ($299, compatible with Model X1)
    |        Extended Battery ($499, 2x runtime)
    |
    |  DATA READ:
    |    Owned attachments  --> Firestore robots/{id}/attachments subcollection
    |    Catalog items      --> Firestore attachment_catalog collection (read-only)
    |  DATA SOURCE: Firestore (both collections)
    |  OFFLINE: Shows cached catalog. Purchase links may not work (external URL).
    |
    +--> [View] on catalog item
    |
    v
/attachments/detail/{catalogId} (AttachmentDetailScreen)
    |  User sees: Full specs, images, price, compatibility, [Buy] link, [Register] button
    |
```

```
    |  DATA READ:
    |    Catalog detail    --> Firestore attachment_catalog/{catalogId}
    |  OFFLINE: Works from cache if previously viewed.
    |
    +--> [Buy] opens external store URL (browser)
    +--> [Register] (user already purchased and physically attached)
    |
    v
/attachments/register (AttachmentRegisterScreen)
    |  User sees: 3-step wizard
    |    Step 1: "Connect the attachment to your robot physically"
    |    Step 2: "Power on the robot"
    |    Step 3: "We'll detect it via BLE" (scanning animation)
    |
    |  Phase F4 (MVP): Manual registration -- user enters serial number or scans QR
    |  Future: BLE auto-detection if attachment has BLE beacon chip
    |
    |  DATA WRITTEN:
    |    - Firestore: robots/{id}/attachments/{auto-id}:
    |        type: "snow_plow"
    |        serialNumber: "ATT-2C4D-6E8F"
    |        name: "Snow Plow Assembly v1"
    |        status: "connected"
    |        registeredAt: <now>
    |        specifications: { plowAngle: { min: -30, max: 30 }, ... }
    |    - Firestore: robots/{id}.attachment updated to reflect new attachment
    |
    |  OFFLINE: Registration requires BLE to robot. No internet needed for the
    |           BLE detection part. Firestore write queues if offline.
```

Journey 5: Offline User -- BLE Direct to Robot

```
SCENARIO: User is in backyard, phone has no WiFi or cellular signal.
Robot is nearby, BLE is available.
OPEN APP
    |
    v
/splash
    |  Firebase Auth token is cached locally -- user is authenticated.
    |  No internet --> Firestore offline cache provides user data.
    |
    v
/dashboard
    |  CONNECTION STATE: BLE Connected (green dot, "Local" label)
    |
    |  WHAT WORKS (all via BLE direct to robot):
    |    [x] View live telemetry (10Hz from BLE GATT characteristics)
    |    [x] View battery status (from telemetry stream)
    |    [x] Manual joystick control (30Hz BLE writes)
    |    [x] Start/stop/pause tasks (BLE command)
    |    [x] E-Stop (BLE command, highest priority)
    |    [x] View zones (from Firestore offline cache / Hive)
    |    [x] Boundary recording (GPS from robot via BLE + local storage)
    |    [x] Blade on/off toggle
    |    [x] View diagnostics (live sensor data from BLE)
    |
    |  WHAT DOES NOT WORK (requires internet):
    |    [ ] Task history (not yet synced to Firestore)
    |    [ ] Push notifications (requires FCM)
    |    [ ] Firmware updates (requires download from Storage)
    |    [ ] Attachment catalog browsing
    |    [ ] Profile changes (queued, sync later)
    |    [ ] Schedule changes propagating to cloud
    |    [ ] Weather data
    |
    |  WHAT PARTIALLY WORKS:
    |    [~] Zone editing -- edits saved to Hive + queued for Firestore
    |    [~] Schedule editing -- edits saved locally, queued for sync
    |    [~] Settings changes -- saved to Hive, queued for Firestore
```

```
    |
    |  WHEN INTERNET RETURNS:
    |    - Firestore SDK automatically syncs all queued writes
    |    - Robot uploads buffered task_history and telemetry_snapshots
    |    - Push notification tokens refresh
    |    - Connection bar changes from "Local" to "Local + Cloud"
```

Journey 6: Cloud User -- Away from Home, Monitor via Cloud

```
SCENARIO: User is at work, 20 miles from robot. Phone has internet.
Robot is at home on WiFi with internet.
OPEN APP
    |
    v
/dashboard
    |  CONNECTION STATE: Cloud (blue dot, "Remote" label)
    |
    |  WHAT WORKS (all via Firebase):
    |    [x] View robot status (Firestore robots/{id}.status)
    |    [x] View telemetry (RTDB, ~5 second delay)
    |    [x] Start/stop tasks via cloud commands (Firestore commands subcollection)
    |    [x] View/edit zones (Firestore, syncs to robot)
    |    [x] View/edit schedules (Firestore, syncs to robot)
    |    [x] View task history with coverage maps
    |    [x] View battery history charts
    |    [x] View diagnostics and alerts
    |    [x] Firmware update initiation
    |    [x] E-Stop via cloud command (200-2000ms latency -- LAST RESORT)
    |    [x] Receive push notifications
    |
    |  WHAT DOES NOT WORK (requires BLE/WiFi):
    |    [ ] Manual joystick control (latency too high for safety)
    |    [ ] Blade toggle (safety: must be local)
    |    [ ] Boundary recording (requires robot GPS via local connection)
    |    [ ] Real-time 10Hz telemetry (only get 0.2Hz via cloud)
    |
    |  CLOUD COMMAND FLOW:
    |    User taps [Start Mowing]
    |        |
    |        v
    |    App creates Firestore: robots/{id}/commands/{auto-id}
    |        type: "start_task", status: "pending"
    |        |
    |        v
    |    Robot is listening (Firestore snapshot listener on commands subcollection)
    |        |
    |        v
    |    Robot reads command, updates status: "acknowledged" (1-3 seconds)
    |        |
    |        v
    |    Robot starts task, updates status: "executing"
    |        |
    |        v
    |    App sees status change via Firestore listener, updates UI
    |        |
    |        v
    |    Robot completes task, updates status: "completed"
    |        |
    |        v
    |    Cloud Function sends push notification
```

Journey 7: Error Flows -- E-Stop, Connection Lost, Robot Stuck

E-Stop Flow

```
USER LONG-PRESSES E-STOP BUTTON (red bar, always visible)
    |
    |  BYPASSES ALL STATE MANAGEMENT -- safety critical path
    |
    v
estopCommandProvider fires simultaneously on ALL channels:
    |
    +---> BLE: Write E-Stop byte to GATT characteristic (immediate, <50ms)
    +---> WiFi: Send E-Stop via WebSocket (immediate, <50ms)
    +---> Firebase: Create command { type: "estop" } in Firestore (200-2000ms)
    |
    v
LOCAL STATE:
    - estopProvider set to TRUE (global, blocks all navigation)
    - All pending commands cancelled
    - Haptic feedback (phone vibrates)
    - UI shows E-Stop active state (red overlay)
    |
    v
ROBOT SIDE:
    - Whichever channel arrives first triggers hardware stop
    - All motors stop immediately (hardware interrupt, not software)
    - Robot state set to "estop"
    - Robot reports estop state via all channels back to app
    |
    v
RECOVERY:
    - User must explicitly clear E-Stop
    - Multi-step unlock: confirm button -> safety check -> clear
    - Robot runs self-diagnostics before clearing
    - App verifies robot is safe before allowing new commands
    |
    DATA WRITTEN:
    - Firestore: robots/{id}/alerts/{auto-id} with severity: "critical"
    - Firestore: robots/{id}.status updated to "estop"
    - Cloud Function: push notification "E-Stop activated"
    - Local: estop event logged with timestamp
```

Connection Lost During Active Task

```
ROBOT IS MOWING, CONNECTION DROPS
    |
    v
CONNECTION TIMEOUT STATE MACHINE (robot-side):
    |
    |  0-10 seconds: Robot continues task, buffers commands
    |  10-20 seconds: Robot pauses task, holds position
    |  20-30 seconds: Robot returns to dock
    |  30+ seconds: Robot emergency stops
    |
    v
APP SIDE:
    |
    |  Connection bar: shows "Reconnecting..." (yellow dot, animated)
    |  Dashboard: shows last known state (cached telemetry)
    |  Retry cycle: attempt BLE scan -> WiFi scan -> cloud auth every 5 seconds
    |
```

```
    +--> Connection restored:
    |       - Telemetry stream resumes
    |       - Robot reports current state
    |       - If robot paused itself, user can resume
    |       - Queued Firestore writes sync
    |
    +--> Connection not restored:
            - After 60 seconds: "Connection Lost" overlay
            - Dashboard shows stale data with timestamp
            - "Retry" button prominent
            - If robot returned to dock autonomously, user sees this
              when they eventually reconnect
```

Robot Stuck

```
ROBOT DETECTS IT CANNOT MOVE (wheels spinning, no position change)
    |
    v
ROBOT-SIDE:
    - Robot creates alert: severity: "critical", category: "motor",
      code: "OBSTACLE_DETECTED" or "WHEELS_STUCK"
    - Robot pauses task
    - Robot reports state: "error" via all channels
    |
    v
APP SIDE (if connected):
    - Dashboard shows "Error" state with red badge
    - Diagnostics screen shows the alert
    - Push notification: "Robot stuck -- manual intervention needed"
    - User options:
        [1] [Retry] -- robot attempts to reverse and re-navigate
        [2] [Return to Dock] -- robot tries to return home
        [3] [Manual Control] -- user takes over with joystick
        [4] [E-Stop] -- always available
    |
    DATA WRITTEN:
    - Firestore: robots/{id}/alerts/{auto-id} (robot writes)
    - Firestore: robots/{id}.status: "error" (robot writes)
    - If task was active: robots/{id}/task_history/{id} with status: "failed"
```

PART 2

SCREEN-BY-DATA MATRIX

Complete Screen Inventory

The app has 17 screens (implemented or planned), organized into 5 bottom nav tabs plus modal/push screens.

Screen 01: AuthScreen (`/auth`)

Aspect	Details
File	`lib/features/auth/presentation/auth_screen.dart`
BLoC/State	`AuthBloc` (lib/features/auth/bloc/auth_bloc.dart)
Orientation	Portrait
Bottom Nav	Hidden
E-Stop	Hidden

Data Displayed	Source	Sync Classification
App logo	Local asset	LOCAL-ONLY
Tagline text	Hardcoded	LOCAL-ONLY
Pre-filled email	Local state (form)	LOCAL-ONLY

User Action	Data Written	Destination	Offline Behavior
Sign In (email/pass)	Auth credentials	Firebase Auth	BLOCKED -- requires internet
Create Account	New user credentials	Firebase Auth + Firestore users/{uid}	BLOCKED -- requires internet
Google Sign In	OAuth token	Firebase Auth	BLOCKED -- requires internet
Apple Sign In	OAuth token	Firebase Auth	BLOCKED -- requires internet
Continue as Guest	Nothing	Local state only	WORKS -- skips auth entirely
Forgot Password	Email address	Firebase Auth (sends reset email)	BLOCKED -- requires internet

Screen 02a: BleScanScreen (`/onboarding/scan`)

Aspect	Details
File	`lib/features/onboarding/presentation/ble_scan_screen.dart`

Aspect	Details
BLoC/State	`OnboardingBloc` + local StatefulWidget state
Orientation	Portrait
Bottom Nav	Hidden
E-Stop	Hidden

Data Displayed	Source	Sync Classification
BLE scanning animation	Local animation	LOCAL-ONLY
Discovered robot name	BLE advertisement data	ROBOT-ONLY
Signal strength bars	BLE RSSI value	ROBOT-ONLY

User Action	Data Written	Destination	Offline Behavior
Wait for scan	None (reads BLE ads)	--	WORKS -- BLE is local
Tap "Pair This Robot"	Selected robot BLE address	Local state	WORKS

Screen 02b: PairingScreen (`/onboarding/pair`)

Aspect	Details
File	`lib/features/onboarding/presentation/pairing_screen.dart`
BLoC/State	`OnboardingBloc` + local state
Orientation	Portrait
Bottom Nav	Hidden
E-Stop	Hidden

Data Displayed	Source	Sync Classification
Robot name	BLE advertisement	ROBOT-ONLY
Robot UUID (CHORE-A1B2-C3D4)	BLE GATT characteristic	ROBOT-ONLY
BLE signal info	BLE RSSI	ROBOT-ONLY
Attachment detected	BLE GATT characteristic	ROBOT-ONLY
Attachment serial	BLE GATT characteristic	ROBOT-ONLY

User Action	Data Written	Destination	Offline Behavior
Enter 6-digit pairing code	Code bytes	Robot via BLE write	WORKS -- fully local
Tap "Pair This Robot"	Pairing confirmation	Robot via BLE	WORKS

Screen 02c: WifiSetupScreen (`/onboarding/wifi`)

Aspect	Details
File	`lib/features/onboarding/presentation/wifi_setup_screen.dart`
BLoC/State	`OnboardingBloc` + local state
Orientation	Portrait

Data Displayed	Source	Sync Classification
Pairing success banner	Local state (pairing result)	LOCAL-ONLY
Available WiFi networks	Robot scans WiFi, sends list via BLE	ROBOT-ONLY
Signal strength per network	Robot WiFi scan results	ROBOT-ONLY
Lock icon (secured network)	Robot WiFi scan results	ROBOT-ONLY

User Action	Data Written	Destination	Offline Behavior
Select network	Selected SSID	Local state	WORKS
Enter WiFi password	SSID + password	Robot via BLE write	WORKS
Tap "Connect Robot to Wi-Fi"	WiFi credentials	Robot via BLE	WORKS
Tap "Skip"	Nothing	Navigate only	WORKS

Screen 02d: SetupCompleteScreen (`/onboarding/complete`)

Aspect	Details
File	`lib/features/onboarding/presentation/setup_complete_screen.dart`
BLoC/State	`OnboardingBloc`
Orientation	Portrait

Data Displayed	Source	Sync Classification
Robot name	BLE session data	LOCAL-ONLY (temp)
Connection mode	Current connection state	LOCAL-ONLY
WiFi network name	WiFi setup result	LOCAL-ONLY
Battery %	Robot telemetry via BLE	ROBOT-ONLY
Attachment name	BLE GATT data	ROBOT-ONLY

User Action	Data Written	Destination	Offline Behavior
Screen loads	Robot document	Firestore robots/{serialNumber}	Queued if offline
Screen loads	robotIds array update	Firestore users/{uid}	Queued if offline
Tap "Go to Dashboard"	Nothing	Navigate to /dashboard	WORKS

Screen 03: DashboardScreen (`/dashboard`)

Aspect	Details
File	`lib/features/dashboard/presentation/dashboard_screen.dart`
BLoC/State	TelemetryBloc, RobotBloc, ConnectionProvider
Orientation	Portrait
Bottom Nav	Visible (Tab 0: Home)
E-Stop	Visible

Data Displayed	Source	Sync Classification
Connection mode (Local/Remote/Offline)	connectionProvider (local state)	LOCAL-ONLY
Battery % + estimated runtime	RTDB telemetry OR BLE telemetry	REQUIRED (RTDB for cloud, BLE for local)
Robot state (Idle/Mowing/etc.)	RTDB telemetry/state OR BLE	REQUIRED
Robot name ("ChoreBot Alpha")	Firestore robots/{id}.name	REQUIRED
Attachment info	Firestore robots/{id}.attachment	REQUIRED
Battery cycles	RTDB telemetry/battery_cycles OR BLE	ROBOT-ONLY
Signal strength	RTDB telemetry/signal_strength OR BLE	ROBOT-ONLY
Blade status	RTDB telemetry/state OR BLE	ROBOT-ONLY
Speed (m/s)	RTDB telemetry/speed_ms OR BLE	ROBOT-ONLY
Temperature	RTDB telemetry/system_temp_c OR BLE	ROBOT-ONLY
LIVE badge	Presence of active telemetry stream	LOCAL-ONLY
Recent activity (last 3 events)	Firestore robots/{id}/task_history	REQUIRED
User initials avatar ("JS")	Firestore users/{uid}.displayName	REQUIRED

User Action	Data Written	Destination	Offline Behavior
Tap "Mow"	Nothing (navigate)	Push /mowing	WORKS
Tap "Plow"	Nothing (navigate)	Push /snow	WORKS
Tap "Collect"	Nothing (navigate)	Push /leaf	WORKS
Tap profile avatar	Nothing (navigate)	Push /profile	WORKS
Tap settings gear	Nothing (navigate)	Push /settings	WORKS
Tap "View All" on activity	Nothing (navigate)	Push /history	WORKS

Screen 04: ControlScreen (`/control`) -- Landscape

Aspect	Details
File	`lib/features/control/presentation/control_screen.dart`
BLoC/State	JoystickCommandProvider, TelemetryStreamProvider
Orientation	Landscape (forced)
Bottom Nav	Hidden
E-Stop	Visible (landscape layout)

Data Displayed	Source	Sync Classification
Connection status + "MANUAL" badge	connectionProvider	LOCAL-ONLY
Speed (m/s) with large readout	BLE/WiFi telemetry (10Hz)	ROBOT-ONLY
Heading (degrees) with large readout	BLE/WiFi telemetry	ROBOT-ONLY
Battery %	BLE/WiFi telemetry	ROBOT-ONLY
Compass widget	BLE/WiFi telemetry heading	ROBOT-ONLY
Motor speeds (FL/FR/RL/RR)	BLE/WiFi telemetry	ROBOT-ONLY
Blade on/off state	Local state + BLE	ROBOT-ONLY

User Action	Data Written	Destination	Offline Behavior
Move joystick	Velocity + heading at 30Hz	Robot via BLE/WiFi ONLY	REQUIRES local connection
Toggle blade on/off	Blade state byte	Robot via BLE/WiFi	REQUIRES local connection
Tap "Auto Mode"	Switch-to-auto command	Robot via BLE/WiFi	REQUIRES local connection
Tap "Return to Dock"	Dock command	Robot via BLE/WiFi (preferred) or Firestore	Cloud fallback available
Tap "Pause"	Pause command	Robot via BLE/WiFi (preferred) or Firestore	Cloud fallback available
Long-press E-Stop	E-Stop on ALL channels	BLE + WiFi + Firestore simultaneously	Works if ANY channel available

CRITICAL: Manual joystick control is NEVER available via cloud. The 200-2000ms latency makes it unsafe.

Screen 05: MapScreen (`/map`) -- Landscape

Aspect	Details
File	`lib/features/map/presentation/map_screen.dart`
BLoC/State	ZonesProvider, TelemetryStreamProvider
Orientation	Landscape (forced)
Bottom Nav	Hidden
E-Stop	Visible (landscape layout)

Data Displayed	Source	Sync Classification
Zone polygons (Mow=green, Snow=blue, Leaf=orange, No-Go=red)	Firestore robots/{id}/zones (cached in Hive)	REQUIRED
Zone info overlay (count per type)	Derived from zones data	REQUIRED
Robot position (blue animated dot)	BLE/WiFi telemetry GPS	ROBOT-ONLY
Dock position	Firestore robots/{id}.dockLocation	REQUIRED
Scale indicator	Computed from zone bounding box	LOCAL-ONLY
Active drawing hint ("Drawing Mow Zone -- 5 pins placed")	Local UI state	LOCAL-ONLY

User Action	Data Written	Destination	Offline Behavior
Select zone type tab (Mow/Snow/Leaf/No-Go)	Nothing	Local state	WORKS
Move joystick (drive along boundary)	Velocity commands at 30Hz	Robot via BLE/WiFi	REQUIRES local connection
Tap "Drop Pin"	Vertex {lat, lng} at robot position	Local state (zone being drawn)	REQUIRES local connection (for GPS)
Tap "Undo"	Remove last vertex	Local state	WORKS
Tap "Done"	Complete zone document	Firestore robots/{id}/zones/{auto-id} + Hive cache	Queued if no internet
Drag existing vertex	Updated vertex coordinates	Firestore zone update (version++)	Queued if no internet

Screen 06: BatteryScreen (`/battery`)

Aspect	Details
File	`lib/features/battery/presentation/battery_screen.dart`
BLoC/State	BatteryProvider (derived from telemetry), BatteryHistoryProvider
Orientation	Portrait
Bottom Nav	Visible (Tab 3: Battery)
E-Stop	Visible

Data Displayed	Source	Sync Classification
Large battery visualization (78%)	RTDB telemetry/battery_pct OR BLE	ROBOT-ONLY
Estimated runtime ("~2h 15m mowing")	Computed from battery % + consumption rate	LOCAL-ONLY (computed)
Charge status badge (Discharging/Charging/Full)	RTDB telemetry/battery_current_amps (sign)	ROBOT-ONLY
Cycle count (142/500)	RTDB telemetry/battery_cycles OR BLE	ROBOT-ONLY
Battery health % (94%)	RTDB telemetry/battery_health_pct OR BLE	ROBOT-ONLY
Temperature (32C)	RTDB telemetry/battery_temp_c OR BLE	ROBOT-ONLY
Voltage (25.2V)	RTDB telemetry/battery_voltage OR BLE	ROBOT-ONLY
Current draw (3.2A)	RTDB telemetry/battery_current_amps OR BLE	ROBOT-ONLY
7-day history chart	Firestore robots/{id}/battery_history (last 7 docs)	REQUIRED

User Action	Data Written	Destination	Offline Behavior
View screen	Nothing (read-only)	--	Shows cached data if no connection

Screen 07: DiagnosticsScreen (`/more/diagnostics`)

Aspect	Details
File	`lib/features/diagnostics/presentation/diagnostics_screen.dart`
BLoC/State	DiagnosticStreamProvider, AlertsProvider
Orientation	Portrait
Bottom Nav	Visible (reached via More tab)
E-Stop	Visible

Data Displayed	Source	Sync Classification
System health summary ("All Systems Normal")	Derived from component states	ROBOT-ONLY (via telemetry)
Tab toggle (Overview/Technical/Faults)	Local UI state	LOCAL-ONLY
Component status rows (Motors, Blade, Battery, Connection, Data Link)	BLE/WiFi diagnostics stream (1Hz)	ROBOT-ONLY
Recent alerts list	Firestore robots/{id}/alerts (query: isActive=true, ordered by createdAt desc)	REQUIRED
Export button	Generates diagnostic report	Local file system

User Action	Data Written	Destination	Offline Behavior
Switch tabs	Nothing	Local state	WORKS
Acknowledge alert	Update acknowledgedAt field	Firestore robots/{id}/alerts/{alertId}	Queued if offline
Export diagnostics	Diagnostic JSON/CSV	Local file	WORKS

Screen 08: MowingScreen (`/mowing`)

Aspect	Details
File	`lib/features/mowing/presentation/mowing_screen.dart`
BLoC/State	MowingConfigProvider, ZonesProvider
Orientation	Portrait
Bottom Nav	Visible
E-Stop	Visible

Data Displayed	Source	Sync Classification
Zone name + area + no-go count	Firestore robots/{id}/zones (Hive cache)	REQUIRED
"Ready" badge	Derived from robot state + battery	ROBOT-ONLY
Pattern options (Straight/Diagonal/Checker/Spiral)	Hardcoded options	LOCAL-ONLY
Stripe angle slider (0-180)	Local form state	LOCAL-ONLY
Pass overlap slider (0-30%)	Local form state	LOCAL-ONLY
Estimated time	Computed from zone area + pattern + speed	LOCAL-ONLY (computed)

Data Displayed	Source	Sync Classification
Battery needed %	Computed from zone area + consumption rate	LOCAL-ONLY (computed)

User Action	Data Written	Destination	Offline Behavior
Select pattern	Nothing	Local state	WORKS
Adjust sliders	Nothing	Local state	WORKS
Tap "Start Mowing"	Start command with full config	BLE/WiFi (preferred) OR Firestore command	REQUIRES some connection

Screen 09: SnowScreen (`/snow`)

Aspect	Details
File	`lib/features/snow/presentation/snow_screen.dart`
BLoC/State	SnowConfigProvider, ZonesProvider
Orientation	Portrait

Data Displayed	Source	Sync Classification
Attachment check card ("Snow Plow Attached")	Firestore robots/{id}.attachment	REQUIRED
Attachment serial (ATT-2C4D-6E8F)	Firestore robots/{id}/attachments	REQUIRED
Start position options (Entrance/Garage/Center/Custom)	Hardcoded	LOCAL-ONLY
Snow depth toggle (Light/Medium/Heavy)	Local state	LOCAL-ONLY
Disposal direction (Left/Right/Pile Zone)	Local state	LOCAL-ONLY
Estimated time + passes	Computed from zone area + snow depth	LOCAL-ONLY

User Action	Data Written	Destination	Offline Behavior
Configure options	Nothing	Local state	WORKS
Tap "Start Plowing"	Start command with plow config	BLE/WiFi or Firestore command	REQUIRES some connection

Screen 10: LeafScreen (`/leaf`)

Aspect	Details
File	`lib/features/leaf/presentation/leaf_screen.dart`
BLoC/State	LeafConfigProvider, ZonesProvider
Orientation	Portrait

Data Displayed	Source	Sync Classification
Attachment check card ("Leaf Collector Attached")	Firestore robots/{id}.attachment	REQUIRED
Mode toggle (Full Zone / Spot Clean)	Local state	LOCAL-ONLY

Data Displayed	Source	Sync Classification
Spot clean mini map	Firestore zones (cached) + local drawing	REQUIRED + LOCAL-ONLY
Collection mode (Mulch / Collect to Bag)	Local state	LOCAL-ONLY
Dump point marker	Local state (set by user)	LOCAL-ONLY
Estimated time + bag capacity	Computed	LOCAL-ONLY

User Action	Data Written	Destination	Offline Behavior
Configure options	Nothing	Local state	WORKS
Draw spot clean area	Vertices	Local state (not persisted)	WORKS
Set dump point	Coordinates	Local state (not persisted)	WORKS
Tap "Start Collection"	Start command with leaf config	BLE/WiFi or Firestore command	REQUIRES some connection

Screen 11: SchedulingScreen (`/scheduling`)

Aspect	Details
File	`lib/features/scheduling/presentation/scheduling_screen.dart`
BLoC/State	ScheduleProvider
Orientation	Portrait

Data Displayed	Source	Sync Classification
Schedule cards list	Firestore robots/{id}/schedules (cached)	REQUIRED
Each card: name, task type, time, days, zone, pattern, enabled toggle	Firestore schedule document fields	REQUIRED
"+" add button	--	LOCAL-ONLY

User Action	Data Written	Destination	Offline Behavior
Toggle schedule on/off	isEnabled: true/false	Firestore robots/{id}/schedules/{id}	Queued if offline
Tap "+" to add	New schedule document	Firestore robots/{id}/schedules/{auto-id}	Queued if offline
Edit schedule	Updated fields	Firestore schedule update	Queued if offline
Delete schedule	Delete document	Firestore schedule delete	Queued if offline

Screen 12: SettingsScreen (`/settings`)

Aspect	Details
File	`lib/features/settings/presentation/settings_screen.dart`
BLoC/State	SettingsProvider, UserBloc
Orientation	Portrait

Aspect	Details
E-Stop	Hidden

Data Displayed	Source	Sync Classification
Robot name + model + serial	Firestore robots/{id}	REQUIRED
Firmware version + "Current"/"Update Available" badge	Firestore robots/{id}.firmwareVersion + firmware collection	REQUIRED
Notification toggles	Firestore users/{uid}.notifications	REQUIRED
Units (metric/imperial)	Firestore users/{uid}.settings.units	REQUIRED
Theme (dark/light/system)	Firestore users/{uid}.settings.theme	REQUIRED
Account email	Firestore users/{uid}.email	REQUIRED
App version	Local (package_info)	LOCAL-ONLY

User Action	Data Written	Destination	Offline Behavior
Rename robot	robots/{id}.name	Firestore	Queued if offline
Change notification prefs	users/{uid}.notifications	Firestore	Queued if offline
Change units	users/{uid}.settings.units	Firestore	Queued if offline
Change theme	users/{uid}.settings.theme	Firestore + local	WORKS (local takes effect immediately)
Sign Out	Clear auth state	Firebase Auth + local state	WORKS

Screen 13: ProfileScreen (`/profile`)

Aspect	Details
File	`lib/features/profile/presentation/profile_screen.dart`
BLoC/State	UserBloc, RobotListProvider
Orientation	Portrait

Data Displayed	Source	Sync Classification
User avatar (initials)	Firestore users/{uid}.displayName	REQUIRED
User name + email	Firestore users/{uid}	REQUIRED
Robot list (name, model, attachment, status)	Firestore robots/{id} for each user.robotIds	REQUIRED
Active/Offline badges per robot	Firestore robots/{id}.status	REQUIRED

User Action	Data Written	Destination	Offline Behavior
Tap "Manage Fleet"	Navigate to /fleet	--	WORKS
Tap "Edit Profile"	Name, display name changes	Firestore users/{uid}	Queued
Tap "Sign Out"	Clear auth state	Firebase Auth + local	WORKS

Screen 14: FleetScreen (`/fleet`)

Aspect	Details
File	`lib/features/fleet/presentation/fleet_screen.dart`
BLoC/State	RobotListProvider, FleetBloc
Orientation	Portrait

Data Displayed	Source	Sync Classification
Robot cards (name, model, serial, attachment, battery, status, firmware, last seen)	Firestore robots/{id} per user.robotIds	REQUIRED
Status indicator (Online/Offline/Charging) per robot	Firestore robots/{id}.status	REQUIRED
"Active" badge on current robot	Local state (activeRobotId)	LOCAL-ONLY
"Add Another Robot" card	--	LOCAL-ONLY

User Action	Data Written	Destination	Offline Behavior
Tap robot card	Set active robotId	Local state	WORKS
Tap "+"	Navigate to /onboarding/scan	--	WORKS

PART 3

DATABASE DESIGN -- WHAT NEEDS FIREBASE

Collection: `users/{userId}`

Aspect	Detail
WHY it exists	Stores user identity, preferences, and robot associations. Enables personalized experience across devices.
WHO writes	App (on sign-up, settings changes)
WHO reads	App
WHEN read	On app launch (cached after), on settings screen open, on profile view
Read pattern	Single document get, then snapshot listener for real-time settings sync

Example document:

```
{
  "email": "jay@chorerobotics.com",
  "displayName": "Jay Shah",
  "createdAt": "2026-03-15T10:30:00Z",
  "updatedAt": "2026-03-24T08:15:00Z",
  "settings": {
    "units": "imperial",
    "temperatureUnit": "fahrenheit",
    "theme": "dark"
  },
  "notifications": {
    "taskComplete": true,
    "taskError": true,
    "batteryLow": true,
    "connectionLost": true,
    "weatherAlert": false,
    "otaAvailable": true,
    "estopTriggered": true
  },
  "robotIds": ["CR-MK1-00001", "CR-MK1-PRO-00002"],
  "accountStatus": "active"
}
```

Queries needed:

- `users/{uid}` -- direct document read (no index needed)

Collection: `robots/{robotId}`

Aspect	Detail
WHY it exists	Central registry of robot identity, ownership, status, and configuration. Required for multi-device access and cloud control.
WHO writes	App (pairing, rename, dock location), Robot (status, firmware version, last seen, attachment, position)
WHO reads	App (dashboard, fleet, settings), Robot (reads its own config)
WHEN read	Dashboard open (snapshot listener), fleet screen, settings screen
Read pattern	Snapshot listener on active robot document. One-time reads for fleet list.

Example document:

```
{
  "serialNumber": "CR-MK1-00001",
  "model": "CHOREROBOT-MK1",
  "ownerId": "aB1cD2eF3gH4",
  "pairedAt": "2026-03-15T14:22:00Z",
  "name": "ChoreBot Alpha",
  "firmwareVersion": "1.0.0",
  "status": "idle",
  "lastSeenAt": "2026-03-24T12:00:00Z",
  "lastKnownPosition": {
    "latitude": 33.6845,
    "longitude": -73.0472
  },
  "attachment": {
    "type": "mower_blade",
    "status": "connected",
    "detectedAt": "2026-03-24T08:00:00Z"
  },
  "wifiConfig": {
    "ssid": "HomeWiFi-5G",
    "configuredAt": "2026-03-15T14:25:00Z"
  },
  "dockLocation": {
    "latitude": 33.6843,
    "longitude": -73.0470
  },
  "createdAt": "2026-03-15T14:22:00Z",
  "updatedAt": "2026-03-24T12:00:00Z"
}
```

Queries needed:

- `robots/{serialNumber}` -- direct document read (serial number IS the document ID)
- No collection-wide queries needed from the app (app reads via user.robotIds)

Subcollection: `robots/{robotId}/zones/{zoneId}`

Aspect	Detail
WHY it exists	Stores the polygonal boundaries that define where the robot operates. Zones must persist across sessions, sync to the robot for autonomous operation, and survive app reinstalls.

Aspect	Detail
WHO writes	App (zone creation, editing, deletion via map screen)
WHO reads	App (map screen, task screens), Robot (downloads zones on connect for autonomous nav)
WHEN read	Map screen open, task screen open, robot boot/reconnect
Read pattern	Collection get (all zones for a robot), cached in Hive after first read. Snapshot listener for updates.

Example document:

```
{
  "name": "Front Yard",
  "type": "mow",
  "color": "#4ADE80",
  "vertices": [
    { "lat": 33.6845, "lng": -73.0475 },
    { "lat": 33.6850, "lng": -73.0470 },
    { "lat": 33.6848, "lng": -73.0465 },
    { "lat": 33.6843, "lng": -73.0468 },
    { "lat": 33.6845, "lng": -73.0475 }
  ],
  "areaSqM": 142.5,
  "perimeterM": 52.3,
  "version": 3,
  "isActive": true,
  "createdAt": "2026-03-16T09:00:00Z",
  "updatedAt": "2026-03-22T15:30:00Z",
  "createdBy": "aB1cD2eF3gH4"
}
```

Queries needed:

- All active zones for a robot: `zones where isActive == true` (index: isActive + type)
- Active zones by type: `zones where isActive == true AND type == "mow"` (composite index)

Subcollection: `robots/{robotId}/schedules/{scheduleId}`

Aspect	Detail
WHY it exists	Stores recurring task schedules so the robot can operate autonomously even without the app or internet.
WHO writes	App (CRUD on scheduling screen)
WHO reads	App (scheduling screen), Robot (reads schedules to know when to start tasks)
WHEN read	Scheduling screen open, robot boot/reconnect
Read pattern	Collection get (all schedules for a robot), cached. Snapshot listener for changes.

Example document:

```
{
  "name": "Sunday Front Yard",
  "taskType": "mow",
  "zoneId": "zone_abc123",
```

```
  "daysOfWeek": [0],
  "startTime": "08:00",
  "timezone": "America/New_York",
  "parameters": {
    "pattern": "straight",
    "angleDeg": 45,
    "overlapPct": 10,
    "cuttingHeightMm": 65
  },
  "isEnabled": true,
  "weatherHold": true,
  "lastExecutedAt": "2026-03-17T08:00:00Z",
  "nextScheduledAt": "2026-03-24T08:00:00Z",
  "createdAt": "2026-03-16T10:00:00Z",
  "updatedAt": "2026-03-20T14:00:00Z",
  "createdBy": "aB1cD2eF3gH4"
}
```

Queries needed:

- Enabled schedules ordered by time: `schedules where isEnabled == true ORDER BY startTime` (composite index)

Subcollection: `robots/{robotId}/commands/{commandId}`

Aspect	Detail
WHY it exists	Enables cloud-based robot control when the user has no direct BLE/WiFi connection. Commands are a queue the robot polls or listens to.
WHO writes	App (creates command), Robot (updates status: acknowledged -> executing -> completed/failed)
WHO reads	Robot (polls/listens for pending commands), App (watches command status)
WHEN read	Robot: continuous Firestore snapshot listener filtered to status=="pending". App: after creating a command, watches its status.
Read pattern	Query: `commands where status == "pending" ORDER BY createdAt`

Example document:

```
{
  "type": "start_task",
  "payload": {
    "taskType": "mow",
    "zoneId": "zone_abc123",
    "parameters": {
      "pattern": "straight",
      "angleDeg": 45,
      "overlapPct": 10,
      "cuttingHeightMm": 65
    }
  },
  "status": "completed",
  "error": null,
  "createdAt": "2026-03-24T08:00:00Z",
  "acknowledgedAt": "2026-03-24T08:00:02Z",
  "executingAt": "2026-03-24T08:00:03Z",
  "completedAt": "2026-03-24T09:20:00Z",
```

```
  "expiresAt": "2026-03-24T08:05:00Z",
  "createdBy": "aB1cD2eF3gH4"
}
```

Queries needed:

- Pending commands: `commands where status == "pending" ORDER BY createdAt` (composite index)
- Expired commands: `commands where status == "pending" AND expiresAt < now` (composite index)
- Old completed: `commands where status IN ["completed","failed","expired"] AND createdAt < 7daysAgo` (for cleanup)

Subcollection: `robots/{robotId}/task_history/{taskId}`

Aspect	Detail
WHY it exists	Records every completed task for user review, analytics, and trend tracking. Users want to know how much area they have mowed, battery trends, and efficiency over time.
WHO writes	Robot (after each task completion)
WHO reads	App (history screen, dashboard recent activity, analytics charts)
WHEN read	Dashboard open (last 5 items), history screen (paginated), analytics screen
Read pattern	Query: `task_history ORDER BY startedAt DESC LIMIT 10` (paginated)

Example document:

```
{
  "taskType": "mow",
  "zoneId": "zone_abc123",
  "zoneName": "Front Yard",
  "scheduleId": null,
  "status": "completed",
  "startedAt": "2026-03-24T08:00:00Z",
  "completedAt": "2026-03-24T09:20:00Z",
  "durationMinutes": 80,
  "activeDurationMinutes": 72,
  "areaCoveredSqM": 135.2,
  "areaTotalSqM": 142.5,
  "coveragePct": 94.9,
  "distanceTraveledM": 1450,
  "batteryStartPct": 78,
  "batteryEndPct": 22,
  "batteryUsedPct": 56,
  "taskResults": {
    "stripesCompleted": 24,
    "cuttingHeightMm": 65,
    "pattern": "straight"
  },
  "failureReason": null,
  "pathSummary": {
    "pointCount": 4800,
    "storageRef": "robots/CR-MK1-00001/coverage/task_xyz.json",
    "boundingBox": {
      "north": 33.6850,
      "south": 33.6843,
      "east": -73.0465,
      "west": -73.0475
```

```
    }
  },
  "createdAt": "2026-03-24T09:20:00Z"
}
```

Queries needed:

- Recent history: `task_history ORDER BY startedAt DESC LIMIT 10`
- History by type: `task_history where taskType == "mow" ORDER BY startedAt DESC`
- History by zone: `task_history where zoneId == "zone_abc" ORDER BY startedAt DESC`
- History by status: `task_history where status == "failed" ORDER BY startedAt DESC`

Subcollection: `robots/{robotId}/battery_history/{entryId}`

Aspect	Detail
WHY it exists	Powers the 7-day battery history chart on the battery screen. Tracks battery health degradation over time.
WHO writes	Robot (once per day at midnight, plus on each charge completion)
WHO reads	App (battery screen chart)
WHEN read	Battery screen open (query last 7 documents)
Read pattern	Query: `battery_history ORDER BY timestamp DESC LIMIT 7`

Example document (document ID: "2026-03-24"):

```
{
  "date": "2026-03-24",
  "timestamp": "2026-03-24T23:59:59Z",
  "batteryPct": 65,
  "voltage": 25.2,
  "currentAmps": -0.1,
  "temperatureC": 22.5,
  "healthPct": 94,
  "cycleCount": 142,
  "dailyStats": {
    "minPct": 18,
    "maxPct": 100,
    "chargeCount": 2,
    "totalRuntimeMinutes": 180,
    "totalChargeMinutes": 240,
    "energyUsedWh": 450
  }
}
```

Subcollection: `robots/{robotId}/alerts/{alertId}`

Aspect	Detail
WHY it exists	Records robot faults and warnings for the diagnostics screen. Enables push notifications on critical events. Provides maintenance history.
WHO writes	Robot (on fault detection), Cloud Functions (on connection loss)
WHO reads	App (diagnostics screen, dashboard activity)
WHEN read	Diagnostics screen open (active alerts query), dashboard (recent alerts)

Example document:

```
{
  "severity": "warning",
  "category": "connection",
  "code": "WIFI_SIGNAL_DROP",
  "title": "Wi-Fi Signal Dropped Briefly",
  "message": "Wi-Fi signal dropped below threshold for 3 seconds. Robot continued via BLE.",
  "component": "wifi_module",
  "telemetrySnapshot": {
    "batteryPct": 72,
    "speedMs": 0.8,
    "position": { "lat": 33.6846, "lng": -73.0471 },
    "systemTempC": 38
  },
  "isActive": false,
  "acknowledgedAt": "2026-03-24T11:00:00Z",
  "resolvedAt": "2026-03-24T10:02:00Z",
  "createdAt": "2026-03-24T10:00:00Z"
}
```

Collection: `firmware/{firmwareId}`

Aspect	Detail
WHY it exists	Tracks available firmware versions so both the app and robot can check for updates. Contains release notes for users and pre-install requirements for safety.
WHO writes	Admin only (Firebase Console or Admin SDK)
WHO reads	App (firmware update screen, settings badge), Robot (periodic update check)
WHEN read	App: settings screen load. Robot: every 6 hours.
Read pattern	Query: `firmware where targetModels array-contains "CHOREROBOT-MK1" AND releaseChannel == "stable" ORDER BY versionCode DESC LIMIT 1`

Realtime Database: `robots/{robotId}/telemetry`

Aspect	Detail
WHY it exists	Provides low-latency, low-cost live telemetry streaming. Firestore charges per-operation; Realtime Database charges per-bandwidth, making it 100x cheaper for continuously-updating data.

Aspect	Detail
WHO writes	Robot (every 5 seconds when cloud-connected)
WHO reads	App (dashboard, battery screen, control screen -- all with real-time listeners)
WHEN read	Continuous `.onValue()` listener whenever app is in foreground
Read pattern	Path listener on `robots/{id}/telemetry` or individual child paths

Example data at path `robots/CR-MK1-00001/telemetry`:

```
{
  "battery_pct": 78,
  "battery_voltage": 25.4,
  "battery_current_amps": -2.1,
  "battery_temp_c": 32.5,
  "battery_health_pct": 94,
  "battery_cycles": 142,
  "speed_ms": 0.8,
  "heading_deg": 127.3,
  "latitude": 33.6845,
  "longitude": -73.0472,
  "rtk_status": "fix",
  "satellite_count": 14,
  "position_accuracy_cm": 2.5,
  "motor_fl_rpm": 1200,
  "motor_fr_rpm": 1180,
  "motor_rl_rpm": 1150,
  "motor_rr_rpm": 1190,
  "system_temp_c": 45.2,
  "state": "working",
  "active_task_id": "task_abc",
  "signal_strength": 3,
  "attachment_type": "mower_blade",
  "timestamp": 1711324800000,
  "_updated_at": 1711324800000
}
```

PART 4

WHAT DOES NOT NEED A DATABASE

Data That is ROBOT-ONLY (never leaves the robot except via BLE/WiFi stream)

Data	Why No Firebase	Where It Lives
Raw sensor readings at 10Hz (IMU, encoders, GPS)	Too high frequency, too high volume, not useful in cloud	Robot RAM, consumed and discarded
Internal PID controller state	Robot-internal control loop, no user visibility	Robot RAM
Motor encoder raw ticks	Low-level hardware data, telemetry sends derived RPM	Robot hardware registers
Camera frames (if FPV enabled)	Too large for Firebase, streamed via WebRTC	Robot GPU buffer
ROS2 topic data	Internal robot pub/sub, app gets summarized telemetry	Robot ROS2 graph
Obstacle detection point cloud	Safety system data, too large and frequent	Robot RAM/GPU

Data That is LOCAL-ONLY (never leaves the phone)

Data	Why No Firebase	Where It Lives
Joystick position (x, y offset)	30Hz UI state, transient, sent to robot not cloud	Flutter widget state
Current tab index in bottom nav	UI navigation state	StatefulWidget `_currentIndex`
Scroll position on any screen	UI state, no persistence value	Flutter ScrollController
Form input text (before submission)	Transient input, only matters on submit	TextEditingController
Animation states (pulse, fade)	Visual presentation only	AnimationController
Selected zone type tab on map	Transient UI selection	Local StatefulWidget
Selected pattern on mowing screen	Transient until "Start" is pressed	Local StatefulWidget
Selected options on snow/leaf screens	Transient until "Start" is pressed	Local StatefulWidget
Theme (after change, before sync)	Applied immediately to UI, synced to Firebase in background	Local SharedPreferences + Firestore
BLE scan results	Ephemeral discovery data	flutter_blue_plus stream

Data That Uses Firestore Offline Cache (OPTIONAL sync, works from cache)

Data	Behavior When Offline	Cache Duration
Zone polygons	Cached in Firestore + Hive, fully readable offline	Indefinite (until explicitly invalidated)
Schedules	Cached in Firestore + Hive, edits queue for sync	Indefinite
User settings	Cached in Firestore + Hive, changes apply locally first	Indefinite
Robot registration data	Cached after first read	Indefinite
Last known telemetry	Cached in Hive (single snapshot)	Until next update
Recent task history (last 20)	Cached in Firestore + Hive	24 hours
Attachment catalog	Cached in Firestore after first browse	Until app restart

Real-Time Control Data (NEVER goes through any database)

Data	Path	Frequency	Why Direct Only
Joystick drive commands	App -> BLE/WiFi -> Robot	30 Hz	33ms budget, Firebase adds 200-2000ms
E-Stop command	App -> BLE + WiFi + Firebase (simultaneously)	On demand	Safety-critical, BLE/WiFi is primary
Blade on/off toggle	App -> BLE/WiFi -> Robot	On demand	Safety: must have local confirmation
Live telemetry stream	Robot -> BLE/WiFi -> App	10 Hz	100ms budget for smooth UI updates
Boundary recording GPS	Robot -> BLE -> App	1-5 Hz	Needs sub-meter accuracy, low latency
Compass heading	Robot -> BLE/WiFi -> App	10 Hz	Part of telemetry stream

PART 5

DATA LIFECYCLE

Zone Polygon Lifecycle

```
BORN: User draws boundary on map screen by driving robot along perimeter
  |   and tapping "Drop Pin" at each vertex
  |
  |   Created as: local array of {lat, lng} pairs in widget state
  |
  v
COMMITTED: User taps "Done" on map screen
  |
  |   Written to:
  |     1. Firestore: robots/{id}/zones/{auto-id} (version: 1)
  |     2. Hive cache: zones_box (for offline reads)
  |   Sync: Robot's Firestore snapshot listener receives the new zone
  |
  v
LIVES: Firestore robots/{id}/zones/{zoneId}
  |     Also: Robot local cache (on-device SQLite), App Hive cache
  |
  v
EDITED: User drags vertex on map screen
  |
  |   Updated in:
  |     1. Firestore: robots/{id}/zones/{zoneId} (version: 2, 3, ...)
  |     2. Hive cache refreshed
  |   Conflict resolution: version field must increment. Stale edits fail.
  |
  v
SOFT-DELETED: User deletes zone
  |
  |   NOT actually deleted. Set isActive: false.
  |   Reason: historical task_history records reference zoneId.
  |   Zone name is denormalized into task_history for this reason.
  |
  v
NEVER HARD-DELETED (by app). Cloud Function may purge after 1 year.
```

Task History Lifecycle

```
BORN: Robot completes (or fails/cancels) a task
  |
  |   Robot writes to Firestore: robots/{id}/task_history/{auto-id}
  |   Robot uploads coverage path to Storage: robots/{id}/coverage/{taskId}.json
  |
  v
LIVES: Firestore robots/{id}/task_history/{taskId}
  |     Coverage path: Firebase Storage (JSON file, 10KB-5MB)
  |     Also cached: App Firestore offline cache + Hive (last 20 items)
```

```
  |
  v
READ: App queries on history screen (paginated, 10 at a time)
  |    Dashboard shows last 5 items
  |    Analytics charts aggregate data
  |
  v
RETAINED: 1 year in Firestore
  |
  v
ARCHIVED/DELETED: Cloud Function deletes records older than 1 year
    Coverage JSON files in Storage cleaned up at same time
```

Live Telemetry Lifecycle

```
BORN: Robot sensors produce data at 10Hz (100ms intervals)
  |   IMU, GPS, encoders, temperature, battery management system
  |
  v
CONSUMED LOCALLY: Robot ROS2 nodes process at 10Hz for control loops
  |
  v
STREAMED TO APP (if local connection):
  |   Via BLE GATT notify characteristic or WiFi WebSocket
  |   Rate: 10Hz (100ms) -- for joystick response and live dashboards
  |
  v
STREAMED TO CLOUD (if internet):
  |   Downsampled to 0.2Hz (every 5 seconds)
  |   Written to Realtime Database: robots/{id}/telemetry (overwrite)
  |   Data size: ~500 bytes per update
  |
  v
SNAPSHOTTED (for history):
  |   Sampled at 1/minute (when active) or 1/5min (when idle)
  |   Written to Firestore: robots/{id}/telemetry_snapshots/{auto-id}
  |
  v
LIVES (Realtime DB): Overwritten continuously. Only "latest" exists.
LIVES (Firestore snapshots): 90 days retention, then TTL-deleted.
  |
  v
DIES:
  - Realtime DB: Overwritten by next update (ephemeral by nature)
  - Firestore snapshots: Deleted after 90 days by Cloud Function
  - BLE/WiFi stream: Consumed by app, never persisted (except Hive "last known")
```

User Settings Lifecycle

```
BORN: User signs up, Firestore user document created with defaults
  |
  v
LIVES: Firestore users/{uid}.settings + users/{uid}.notifications
  |     Also cached in: Hive settings_box, Firestore offline cache
  |
  v
EDITED: User changes setting on settings screen
  |
  |   1. Applied locally immediately (theme change takes effect instantly)
  |   2. Written to Hive (for next app launch before Firestore loads)
```

```
  |   3. Written to Firestore (syncs across devices, queues if offline)
  |
  v
DIES: Only when account is deleted (soft delete via accountStatus: "deleted")
```

Cloud Command Lifecycle

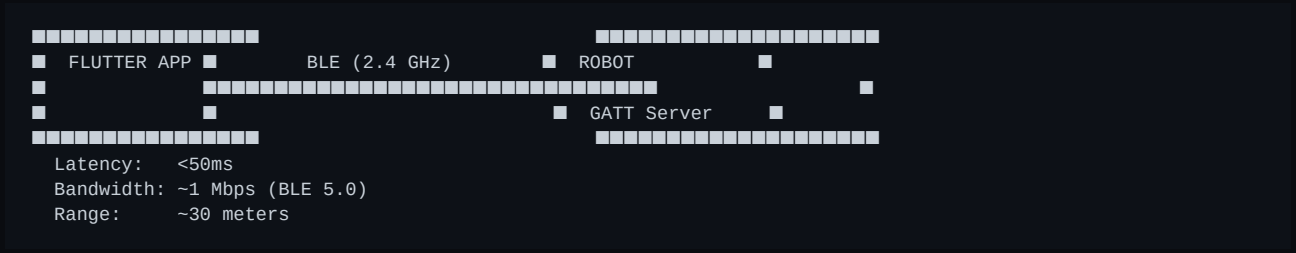
```
BORN: User taps "Start Mowing" while in cloud-only mode
  |
  |   App creates Firestore: robots/{id}/commands/{auto-id}
  |   status: "pending", expiresAt: createdAt + 5 minutes
  |
  v
ACKNOWLEDGED: Robot's Firestore listener fires, robot reads command
  |   Robot updates: status: "acknowledged", acknowledgedAt: now
  |   Latency: 1-5 seconds from creation
  |
  v
EXECUTING: Robot begins task
  |   Robot updates: status: "executing", executingAt: now
  |   App listener sees status change, updates UI
  |
  v
COMPLETED or FAILED: Robot finishes or encounters error
  |   Robot updates: status: "completed" or "failed"
  |   If failed: error: { code: "BATTERY_TOO_LOW", message: "..." }
  |
  v
EXPIRED (alternative path): Robot never acknowledged within 5 minutes
  |   Cloud Function sets status: "expired"
  |   App shows "Command timed out" to user
  |
  v
DIES: Cloud Function deletes completed/failed/expired commands after 7 days
```

PART 6

CONNECTION MODES & DATA ROUTING

Mode 1: BLE Direct (Robot Hotspot)

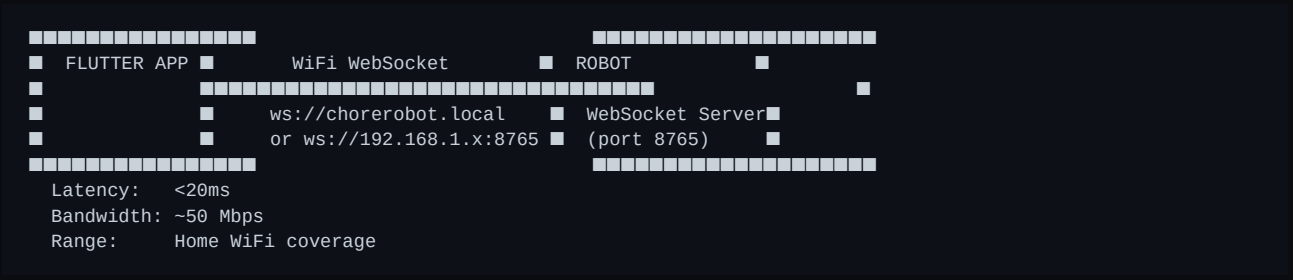
Scenario: User is near robot, phone discovers robot via Bluetooth.



Feature	Works?	Notes
Manual joystick control (30Hz)	YES	Primary use case for BLE
Live telemetry (10Hz)	YES	Via GATT notify characteristics
E-Stop	YES	Highest priority, immediate
Start/stop/pause tasks	YES	Command sent via BLE write
Blade toggle	YES	BLE write to blade characteristic
Boundary recording	YES	GPS from robot via BLE
Zone viewing	YES	From Firestore cache / Hive
Zone editing (vertices)	YES	Needs robot GPS via BLE for "Drop Pin"
Schedule viewing	YES	From Firestore cache / Hive
Schedule editing	PARTIAL	Edits saved locally, queued for Firestore
Task history	PARTIAL	Last 20 from cache, no new data from cloud
Battery history chart	PARTIAL	Cached chart, no new data points
Firmware updates	NO	Requires internet download
Push notifications	NO	Requires FCM (internet)
Cloud commands	NO	Not needed (BLE is faster)
Attachment catalog	NO	Requires internet

Mode 2: WiFi LAN

Scenario: Robot is on home WiFi. Phone is on same WiFi. No internet needed.

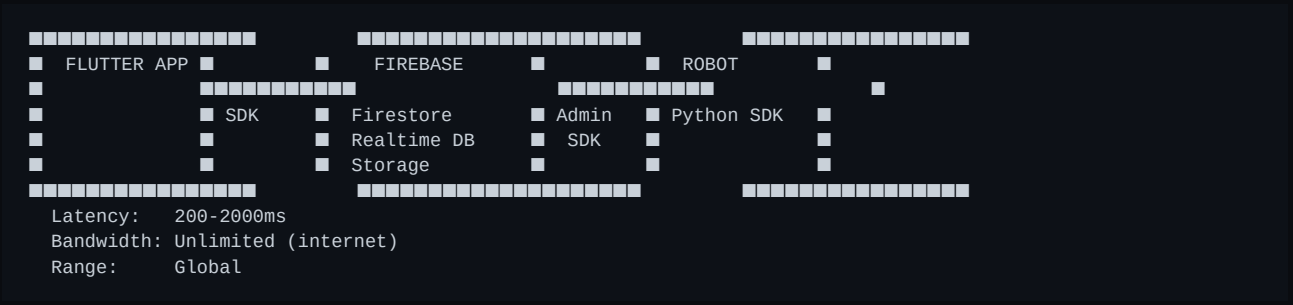


Feature	Works?	Notes
Manual joystick control (30Hz)	YES	WebSocket, lower latency than BLE
Live telemetry (10Hz)	YES	WebSocket, higher bandwidth than BLE
E-Stop	YES	Immediate via WebSocket
Start/stop/pause tasks	YES	WebSocket command
Blade toggle	YES	WebSocket command
Boundary recording	YES	GPS from robot via WebSocket
Zone sync to robot	YES	Can push zones directly over WebSocket
Schedule sync to robot	YES	Can push schedules over WebSocket
Task history	PARTIAL	From cache only
Firmware updates	NO	Requires internet for download
Push notifications	NO	Requires internet

WiFi advantage over BLE: Higher bandwidth enables future features like FPV camera streaming and faster zone data transfer.

Mode 3: Cloud (Firebase)

Scenario: User is away from home. Phone has internet. Robot has internet.

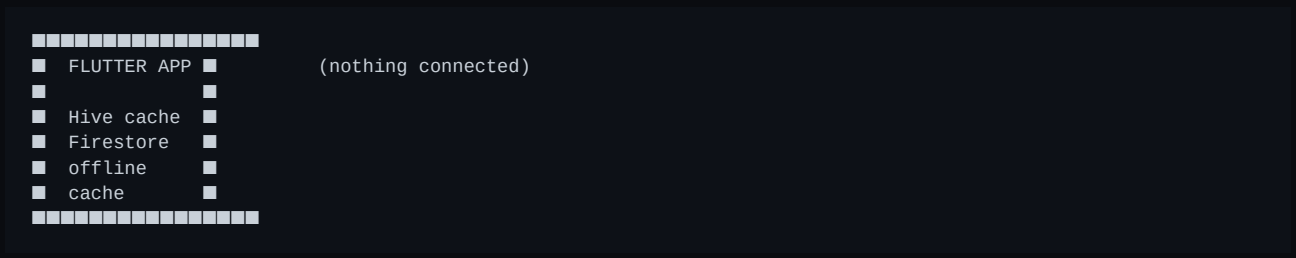


Feature	Works?	Notes
View robot status	YES	Firestore snapshot listener
View telemetry (0.2Hz)	YES	RTDB listener, 5-second updates
Cloud commands (start/stop/dock)	YES	Firestore commands subcollection

Feature	Works?	Notes
E-Stop	YES (last resort)	200-2000ms latency, use BLE/WiFi if possible
Zone viewing	YES	Firestore with offline cache
Zone editing	YES	Firestore write, syncs to robot
Schedule editing	YES	Firestore write, syncs to robot
Task history	YES	Full history with pagination
Battery history chart	YES	Full 7-day + data
Diagnostics / alerts	YES	Firestore queries
Firmware updates	YES	Initiate via cloud command + Storage download
Push notifications	YES	FCM delivers to phone
Manual joystick	NO	Latency too high for safety
Blade toggle	NO	Safety: requires local confirmation
Boundary recording	NO	Requires robot GPS via local connection

Mode 4: Offline (No Connection At All)

Scenario: No BLE, no WiFi, no internet. Phone is a brick.



Feature	Works?	Notes
View cached dashboard	YES	Last known state from Hive
View cached zones	YES	From Hive zones_box
View cached schedules	YES	From Hive schedules_box
View cached settings	YES	From Hive settings_box
View recent history (last 20)	YES	From Hive history_box
Edit zones (saved locally)	YES	Queued for Firestore sync
Edit schedules (saved locally)	YES	Queued for Firestore sync
Any robot control	NO	No communication path
Any live telemetry	NO	No communication path
E-Stop	NO	No communication path
Navigation between screens	YES	App is fully navigable
"Offline" banner visible	YES	Shows prominently

Feature	Works?	Notes
Connection retry	YES	Every 5 seconds, BLE scan + WiFi check

PART 7

SECURITY & PERMISSIONS

Who Can Read/Write What (Human-Readable Summary)

Users Collection

Operation	Who	Rule
Read own profile	Authenticated user whose UID matches	Only you can see your data
Create profile	Authenticated user, email must match auth token	Prevents impersonation
Update profile	Owner only, cannot change email or createdAt	Immutable identity fields
Delete profile	Nobody (soft delete via accountStatus)	Data retention
Read FCM tokens	Owner only	Your devices only
Write FCM tokens	Owner only	Register your own devices
Read notification log	Owner only	Your notifications
Write notification log	Cloud Functions / Robot service account only	System-generated
Mark notification read	Owner only (can only update `read` field)	Limited write scope

Robots Collection

Operation	Who	Rule
Read robot data	Owner (ownerId matches) OR Robot itself OR Admin	Data isolation
Create robot	Authenticated user, ownerId must be self	Register during pairing
Update (rename, dock, wifi)	Owner only, restricted field set	Users can customize
Update (status, firmware, position)	Robot service account only, restricted field set	Robot reports its own state
Delete robot	Admin only	Prevent accidental deletion

Robot Subcollections

Subcollection	Who Reads	Who Writes	Special Rules
zones	Owner + Robot	Owner (create/update/delete)	Version must increment on update. Min 3, max 1000 vertices. Type must be valid enum.
schedules	Owner + Robot	Owner (CRUD)	taskType must be valid. daysOfWeek 1-7 elements.
commands	Owner + Robot	Owner creates (status must be "pending"). Robot updates status only (valid transitions enforced).	Commands never deleted by clients.
task_history	Owner only	Robot only (create). Immutable after creation.	No updates, no deletes.
battery_history	Owner only	Robot only	Append/overwrite by date
alerts	Owner only	Robot creates. Owner can acknowledge (update acknowledgedAt only). Robot can resolve (update isActive + resolvedAt).	Fine-grained field-level permissions
telemetry_snapshots	Owner only	Robot only (create). Immutable.	Append-only log

Firmware Collection

Operation	Who	Rule
Read	Any authenticated user or robot	Everyone checks for updates
Write	Admin only	Controlled release process

Realtime Database

Path	Who Reads	Who Writes
robots/{id}/telemetry	Owner (via _robot_owners lookup) + Robot + Admin	Robot + Admin only
robot_status/{id}	Same as telemetry	Robot + Admin only
_robot_owners/{id}	Any authenticated (lookup table)	Admin only (Cloud Function)

Robot Authentication

The robot runs Firebase Admin SDK with a service account provisioned during manufacturing.

For Firestore: The Admin SDK bypasses all security rules (full access). The robot code is trusted and only writes to its own paths.

For Realtime Database: The robot uses a custom auth token with claims `{ robot_service_account: true, robot_serial: "CR-MK1-00001" }`. Realtime Database rules check these claims.

Security tradeoff: Admin SDK bypass is acceptable for MVP because the robot is trusted hardware we manufacture. In production (Go + PostgreSQL), every robot request is authenticated via mTLS certificates and authorized per-endpoint.

User Data Isolation

User A cannot see User B's robots because:

1. Robot documents have `ownerId` field checked in security rules
2. User documents store their own `robotIds` array
3. All subcollection reads check robot ownership first
4. Realtime Database uses `_robot_owners` lookup table to verify ownership
5. Storage uses signed URLs generated by Cloud Functions (which verify ownership)

PART 8

ATTACHMENT SYSTEM

Attachment Types

Attachment	Type ID	Use Case	Key Specs	Task Type
Mower Blade Assembly	`mower_blade`	Lawn mowing	bladeWidth, bladeHeight (min/max mm)	mow
Snow Plow	`snow_plow`	Snow clearing	plowWidth, plowAngle (min/max deg), plowWeight	plow
Leaf Collector	`leaf_collector`	Leaf collection	bagCapacity (L), vacuumPower, nozzleWidth	collect
Edge Trimmer	`edge_trimmer`	Precision edging	trimWidth, lineLength	mow (variant)
Extended Battery	`extended_battery`	Longer runtime	capacityAh, voltage, weight	All tasks

How Attachments Are Registered

Current (Phase F4 MVP): Manual Registration

```
User physically connects attachment to robot
    |
    v
User opens /attachments/register in app
    |
    v
Step 1: "Connect the attachment to your robot"
    |   User confirms physical connection
    |
    v
Step 2: "Power on the robot"
    |   Robot detects attachment via hardware pins (not BLE)
    |   Robot reports attachment type via BLE GATT characteristic
    |
    v
Step 3: User enters attachment serial number
    |   (printed on attachment label, format: ATT-XXXX-XXXX)
    |   OR scans QR code on attachment
    |
    v
App writes to Firestore:
    robots/{id}/attachments/{auto-id}:
      type: "snow_plow"
```

```
      serialNumber: "ATT-2C4D-6E8F"
      name: "Snow Plow Assembly v1"
      status: "connected"
      registeredAt: <now>
      specifications: { ... type-specific specs ... }
App updates Firestore:
    robots/{id}.attachment:
      type: "snow_plow"
      status: "connected"
      detectedAt: <now>
```

Future: BLE Auto-Detection

```
Attachment has embedded BLE beacon chip
    |
    v
Robot detects attachment BLE MAC address on physical connection
    |
    v
Robot reports to app via BLE: "Attachment detected: ATT-2C4D-6E8F, type: snow_plow"
    |
    v
App auto-registers (no manual serial entry needed)
    |
    v
Same Firestore writes as manual flow
```

How the App Knows Which Attachment Is Connected

Three sources, in priority order:

1. **BLE GATT characteristic** (most authoritative, real-time):

Robot exposes a GATT characteristic that reports the currently connected attachment type and serial number. The app reads this on BLE connection and subscribes for changes.

2. **Firestore robots/{id}.attachment** (for cloud mode):

The robot updates this field whenever attachment state changes. The app reads it when not locally connected.

3. **Firestore robots/{id}/attachments subcollection** (for history):

All registered attachments for the robot. Shows which attachments the user owns, even if not currently connected.

Attachment-Specific Settings Per Task Type

Task Screen	Reads From	Attachment Data Used
Mowing (`/mowing`)	`robots/{id}/attachments` where type=="mower_blade"	bladeHeight min/max (sets slider range), bladeWidth (affects time estimate)
Snow (`/snow`)	`robots/{id}/attachments` where type=="snow_plow"	plowAngle range, plowWidth (affects passes estimate)
Leaf (`/leaf`)	`robots/{id}/attachments` where type=="leaf_collector"	bagCapacity (affects dump estimates), vacuumPower level

Attachment Maintenance Tracking

Each attachment document in Firestore includes a `maintenanceLog` array:

```
{
  "type": "mower_blade",
  "serialNumber": "ATT-7F3E-9A1B",
  "maintenanceLog": [
    {
      "date": "2026-03-20T10:00:00Z",
      "type": "blade_replace",
      "notes": "Replaced dulled blade after 50 hours of use"
    },
    {
      "date": "2026-03-10T14:00:00Z",
      "type": "cleaning",
      "notes": "Cleaned grass buildup from under deck"
    },
    {
      "date": "2026-02-28T09:00:00Z",
      "type": "calibration",
      "notes": "Calibrated blade height sensor"
    }
  ]
}
```

Maintenance types: `blade_replace`, `blade_sharpen`, `cleaning`, `calibration`, `lubrication`, `inspection`, `repair`, `other`.

Attachment Catalog (Read-Only)

The `attachment_catalog` Firestore collection is admin-managed and provides the "store" experience:

```
{
  "type": "edge_trimmer",
  "name": "ChoreBot Edge Trimmer v1",
  "description": "Precision edge trimming for sidewalks, driveways, and garden borders.",
  "imageUrl": "https://storage.googleapis.com/.../edge_trimmer.jpg",
  "price": 299.99,
  "currency": "USD",
  "compatibility": ["CHOREROBOT-MK1", "CHOREROBOT-MK1-PRO"],
  "specifications": {
    "trimWidth": 25,
    "lineLength": 500,
    "weight": 2.3
  },
  "storeUrl": "https://chorerobotics.com/store/edge-trimmer",
  "available": true
}
```

PART 9

OFFLINE STRATEGY DECISION TREE

Master Decision Tree

```
User wants to perform [action]
|
+-- Is this a SAFETY action (E-Stop)?
|   |
|   +-- YES -->  Send on ALL available channels simultaneously
|   |            BLE (if connected): immediate
|   |            WiFi (if connected): immediate
|   |            Firebase (if internet): 200-2000ms
|   |            If NO channels available: E-Stop BUTTON ON ROBOT (hardware)
|   |
|   +-- NO --> Continue below
|
+-- Is this a REAL-TIME CONTROL action (joystick, blade)?
|   |
|   +-- YES --> BLE or WiFi connected?
|   |           |
|   |           +-- YES --> Send command directly to robot
|   |           |           (30Hz joystick, immediate blade toggle)
|   |           |
|   |           +-- NO --> ACTION BLOCKED
|   |                      Show: "Connect to robot for manual control"
|   |                      Reason: Cloud latency (200-2000ms) is unsafe
|   |
|   +-- NO --> Continue below
|
+-- Is this a TASK COMMAND (start, stop, pause, dock)?
|   |
|   +-- YES --> BLE or WiFi connected?
|   |           |
|   |           +-- YES --> Send via BLE/WiFi (preferred, <50ms)
|   |           |
|   |           +-- NO --> Internet available?
|   |                      |
|   |                      +-- YES --> Send via Firestore cloud command
|   |                      |           (200-2000ms, robot polls/listens)
|   |                      |
|   |                      +-- NO --> ACTION BLOCKED
|   |                                 Show: "No connection to robot"
|   |
|   +-- NO --> Continue below
|
+-- Is this a DATA READ (dashboard, history, zones, battery)?
|   |
|   +-- YES --> What data?
|               |
|               +-- Live telemetry (speed, heading, battery real-time):
|               |   |
|               |   +-- BLE/WiFi? --> Stream at 10Hz from robot
|               |   +-- Internet? --> Stream at 0.2Hz from RTDB
|               |   +-- Nothing? --> Show last cached value + stale timestamp
|               |
|               +-- Persistent data (zones, schedules, history, settings):
```

```
|               |   |
|               |   +-- In Hive cache? --> Use it (fastest)
|               |   +-- In Firestore offline cache? --> Use it
|               |   +-- Internet available? --> Fetch fresh from Firestore
|               |   +-- Nothing cached? --> Show empty state
|               |
|               +-- Battery history chart:
|                   |
|                   +-- Firestore cache has data? --> Show cached chart
|                   +-- Internet? --> Query battery_history, last 7 days
|                   +-- Nothing? --> Show "No data available" placeholder
|
+-- Is this a DATA WRITE (zone edit, schedule change, settings)?
    |
    +-- YES --> Write to Hive cache FIRST (immediate local effect)
                |
                +-- Internet available?
                |   |
                |   +-- YES --> Write to Firestore directly
                |               Firestore SDK handles it
                |
                |   +-- NO --> Firestore SDK QUEUES the write
                |              Write will sync automatically when
                |              internet returns. No user action needed.
                |              Show subtle "Changes will sync" indicator.
                |
                +-- Zone edit specifically:
                    |
                    +-- Version field increments on each edit
                    +-- If two devices edit same zone offline:
                        Last-to-sync wins (version conflict detected)
                        App re-fetches and prompts user to reconcile
```

Per-Data-Type Offline Strategies

Data Type	Offline Read	Offline Write	Sync Strategy
User profile	Firestore cache + Hive	Queued (Firestore SDK)	Last-write-wins
Robot registration	Firestore cache + Hive	Queued (Firestore SDK)	Last-write-wins
Zones	Hive cache (primary) + Firestore cache	Queued + version conflict detection	Version field, re-fetch on conflict
Schedules	Hive cache + Firestore cache	Queued (Firestore SDK)	Last-write-wins
Task history	Hive (last 20) + Firestore cache	N/A (robot writes only)	Robot buffers and uploads on reconnect
Battery history	Firestore cache	N/A (robot writes only)	Robot writes daily, cached by Firestore
Alerts	Firestore cache	Acknowledgment queued	Last-write-wins
Live telemetry	Hive (last snapshot only)	N/A (robot sends only)	Ephemeral, no persistence needed
Cloud commands	N/A (cloud-only feature)	Queued (Firestore SDK)	Expires after 5 minutes
Firmware metadata	Firestore cache	N/A (admin writes only)	Cached, checked every 6 hours
Attachment data	Firestore cache	Queued (Firestore SDK)	Last-write-wins
Settings	Hive (instant) + Firestore cache	Hive (instant) + Firestore queued	Hive is source of truth for UI, Firestore syncs

Firestore Offline Cache Configuration

```
// Configured in main.dart during Firebase initialization
FirebaseFirestore.instance.settings = const Settings(
  persistenceEnabled: true,         // Default on mobile, explicit for clarity
  cacheSizeBytes: 40 * 1024 * 1024, // 40 MB local cache (default: 100 MB)
);
```

Why 40 MB instead of 100 MB:

- Most data is small (zones, schedules, settings < 100 KB total)
- Large data (coverage paths) is in Storage, not Firestore
- 40 MB prevents the cache from growing unbounded on older phones
- Firestore SDK auto-evicts least-recently-used cached documents when full

Hive Cache Boxes

Box Name	Contents	Update Trigger	Max Size
`robot`	Active robot config (name, serial, model, firmware)	On robot selection change, on Firestore update	~1 KB
`zones`	All zones for active robot	On zone CRUD, on Firestore listener update	~50 KB
`schedules`	All schedules for active robot	On schedule CRUD, on Firestore update	~10 KB
`telemetry`	Last known telemetry snapshot	On every telemetry update (overwritten)	~500 bytes
`settings`	User preferences (units, theme, notifications)	On settings change	~500 bytes
`presets`	Saved task configurations (Phase 2)	On preset CRUD	~5 KB
`history`	Recent 20 task history records	On task completion notification	~20 KB

APPENDIX

Complete File-to-Screen Mapping

Route	Screen File	BLoC/State File
/auth	lib/features/auth/presentation/auth_screen.dart	lib/features/auth/bloc/auth_bloc.dart
/onboarding/scan	lib/features/onboarding/presentation/ble_scan_screen.dart	lib/features/onboarding/bloc/onboarding_bloc.dart
/onboarding/pair	lib/features/onboarding/presentation/pairing_screen.dart	lib/features/onboarding/bloc/onboarding_bloc.dart
/onboarding/wifi	lib/features/onboarding/presentation/wifi_setup_screen.dart	lib/features/onboarding/bloc/onboarding_bloc.dart
/onboarding/complete	lib/features/onboarding/presentation/setup_complete_screen.dart	lib/features/onboarding/bloc/onboarding_bloc.dart
/dashboard	lib/features/dashboard/presentation/dashboard_shell.dart -> dashboard_screen.dart	TelemetryBloc, RobotBloc, ConnectionProvider
/control	lib/features/control/presentation/control_screen.dart	JoystickCommandProvider, TelemetryStreamProvider
/map	lib/features/map/presentation/map_screen.dart	ZonesProvider, TelemetryStreamProvider
/battery	lib/features/battery/presentation/battery_screen.dart	BatteryProvider, BatteryHistoryProvider
/more/diagnostics	lib/features/diagnostics/presentation/diagnostics_screen.dart	DiagnosticStreamProvider, AlertsProvider
/mowing	lib/features/mowing/presentation/mowing_screen.dart	MowingConfigProvider
/snow	lib/features/snow/presentation/snow_screen.dart	SnowConfigProvider
/leaf	lib/features/leaf/presentation/leaf_screen.dart	LeafConfigProvider
/scheduling	lib/features/scheduling/presentation/scheduling_screen.dart	ScheduleProvider
/settings	lib/features/settings/presentation/settings_screen.dart	SettingsProvider, UserBloc
/profile	lib/features/profile/presentation/profile_screen.dart	UserBloc, RobotListProvider
/fleet	lib/features/fleet/presentation/fleet_screen.dart	RobotListProvider, FleetBloc

APPENDIX

Firestore Index Requirements

Query	Collection	Fields	Index Type
Pending commands by time	commands	status ASC, createdAt ASC	Composite
Expired commands	commands	status ASC, expiresAt ASC	Composite
Task history by type + date	task_history	taskType ASC, startedAt DESC	Composite
Task history by status + date	task_history	status ASC, startedAt DESC	Composite
Task history by zone + date	task_history	zoneId ASC, startedAt DESC	Composite
Active alerts by severity	alerts	isActive ASC, severity ASC, createdAt DESC	Composite
Active alerts by category	alerts	isActive ASC, category ASC, createdAt DESC	Composite
Active zones by type	zones	isActive ASC, type ASC	Composite
Enabled schedules by time	schedules	isEnabled ASC, startTime ASC	Composite
Unread notifications	notification_log	read ASC, createdAt DESC	Composite
Firmware by type + channel + version	firmware	artifactType ASC, releaseChannel ASC, versionCode DESC	Composite

Disabled indexes (large fields never queried):

- zones.vertices
- task_history.taskResults
- telemetry_snapshots.motors
- firmware.artifact

APPENDIX

Data Retention Policies

Data	Retention	Cleanup Method
User documents	Indefinite (soft delete)	Manual admin action
Robot documents	Indefinite	Admin deletion only
Zones	Indefinite (soft delete via isActive)	Cloud Function purge after 1 year inactive
Schedules	Indefinite	User deletion
Commands (completed/failed/expired)	7 days	Cloud Function (hourly)
Task history	1 year	Cloud Function (daily)
Battery history	90 days	Cloud Function (daily)
Alerts (resolved)	30 days	Cloud Function (daily)
Telemetry snapshots	90 days	Firestore TTL policy
Notification log (read)	90 days	Cloud Function (daily)
FCM tokens (stale)	90 days	Cloud Function (on send failure)
Coverage maps (Storage)	1 year	Cloud Function (matches task_history cleanup)
Firmware binaries (Storage)	Indefinite (last 5 versions)	Admin cleanup

APPENDIX

Cost Summary (Firebase Free Tier)

Service	Free Tier	Our Usage (1 robot)	Fits Free Tier?
Auth	50K MAU	~10 logins/month	YES
Firestore reads	50K/day	~350/day	YES
Firestore writes	20K/day	~1,800/day	YES
Realtime DB bandwidth	10 GB/month	~5.2 MB/day = 156 MB/month	YES
Storage	5 GB	~515 MB	YES
Storage downloads	1 GB/day	~50 MB/day	YES
Cloud Functions	2M invocations/month	~3K/month	YES
FCM	Unlimited	~50/day	YES

Bottleneck at scale: Firestore writes exceed free tier at ~11 robots. First paid cost: ~$0.50/month for 5 robots, ~$10/month for 100 robots. Migration to production stack (Go + PostgreSQL + EMQX) recommended at 100+ robots.